

ASSIGNMENT REPORT

Digital Design

UART Transmitter (UART_TX_TOP) - Configurable-Parity Serial Transmitter

Course	Digital Design
Assignment	UART Transmitter (UART_TX_TOP)
Student Name	Mohamed Hossam El-Sawy
Instructor / TA	Eng. Ali Temsah
Submission Date	August 6, 2026

01 OBJECTIVE

A Universal Asynchronous Receiver/Transmitter (UART) is a full-duplex serial communication block that converts parallel data to/from a serial bit stream. This assignment implements only the transmit half, UART_TX_TOP: it accepts an 8-bit parallel byte, frames it with a start bit, the 8 data bits (LSB first), an optional configurable parity bit, and a stop bit, then shifts the frame out one bit per clock on TX_OUT while asserting Busy.

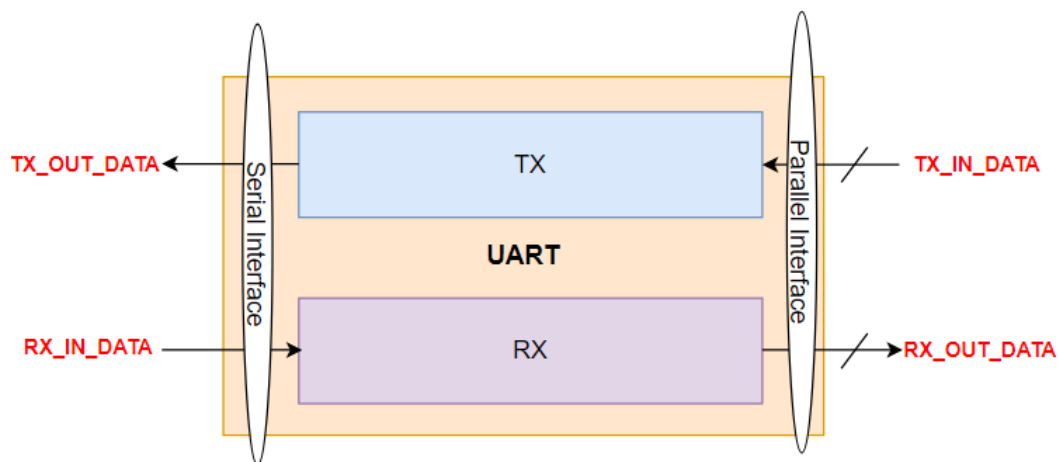


Figure 1: Serial vs. parallel interface of a UART - this assignment implements the TX (serial-out) half

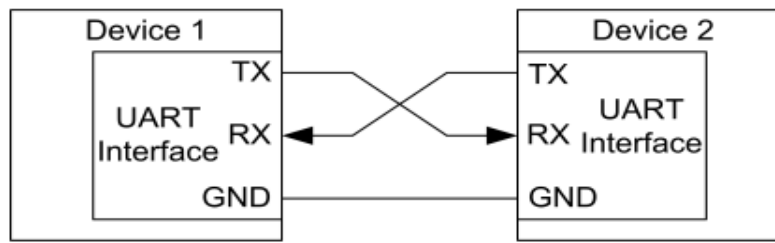


Figure 2: Two UARTs connected TX-to-RX, full duplex

02 PROBLEM STATEMENT / REQUIREMENTS

- P_DATA is only sampled while DATA_VALID is high; DATA_VALID is asserted for exactly one clock cycle per frame.
- All registers are cleared using an asynchronous active-low reset.
- Busy is high for the entire duration of a frame (start bit through stop bit) and low otherwise.
- UART_TX must not accept new P_DATA while a frame is already in progress, even if DATA_VALID happens to pulse again.
- TX_OUT idles high when no frame is being transmitted.
- PAR_EN configures whether a parity bit is included in the frame (0 = no parity, 1 = parity included).
- PAR_TYP configures the parity polarity when PAR_EN=1 (0 = even parity, 1 = odd parity).
- Each frame is: 1 start bit (0) + 8 data bits (LSB first) + optional 1 parity bit + 1 stop bit (1).
- Implement UART_TX in Verilog and verify it with a testbench running at a 200 MHz clock.

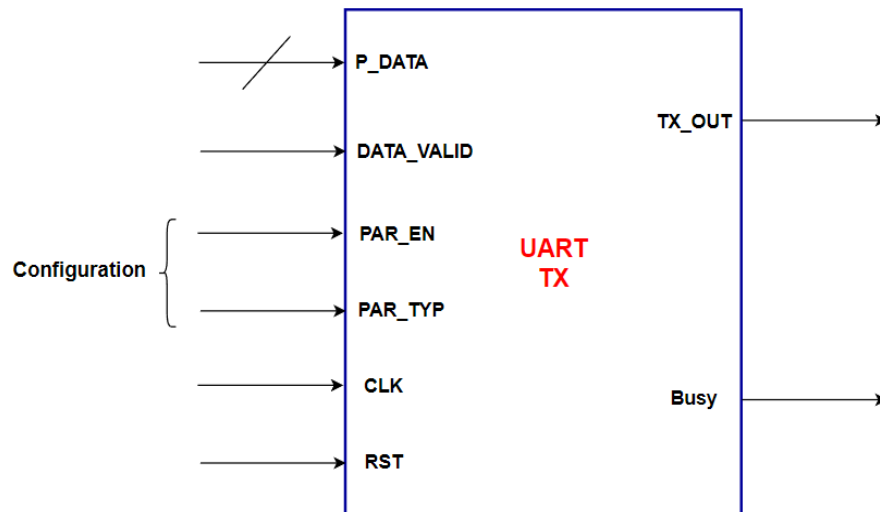


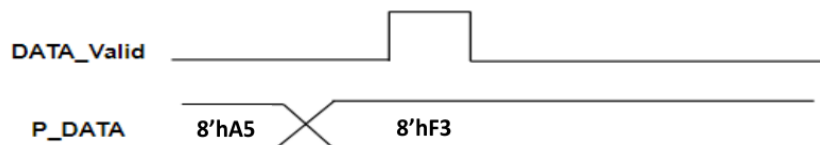
Figure 3: UART_TX block interface as given in the specification - CLK/RST/PAR_TYP/PAR_EN/P_DATA/DATA_VALID in, TX_OUT/Busy out

Port	Direction	Width	Description
P_DATA	input	8	Parallel input data byte
DATA_VALID (dataValid)	input	1	Pulses high for exactly 1 clock to load a new byte
PAR_EN (parEn)	input, configuration	1	0 = no parity bit in the frame, 1 = parity bit included
PAR_TYP (parTyp)	input, configuration	1	0 = even parity, 1 = odd parity
CLK	input	1	System clock (200 MHz in the testbench)
RST	input	1	Asynchronous active-low reset

Port	Direction	Width	Description
TX_OUT (txOut)	output	1	Serial output line - idles high, low start bit, LSB-first data, optional parity bit, high stop bit
Busy	output	1	High for the entire duration of a frame, low when idle

Table 1: UART_TX port list, as specified in the assignment document

Expected Input: -



Expected Output: -

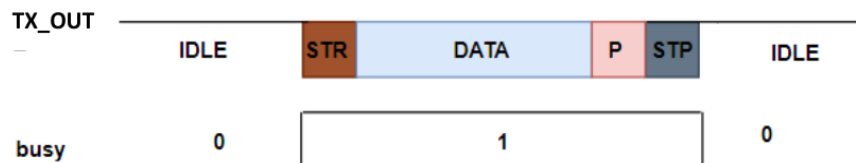
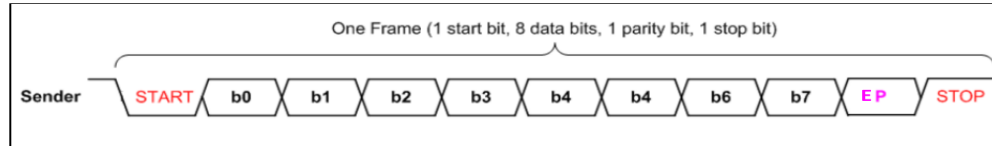


Figure 4: Specification's expected-behaviour waveform - **DATA_VALID** pulses once to load **8'hA5**, then **TX_OUT** sequences **IDLE-STR-DATA-P-STP-IDLE** while **Busy** is high

All Expected Output Frames: -

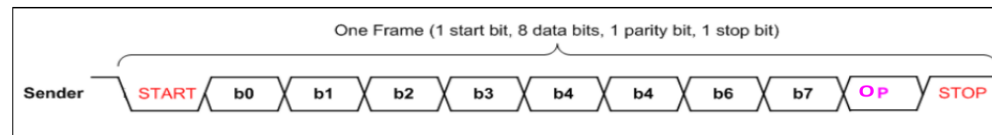
1. Data Frame (in case of Parity is enabled & Parity Type is even)

- One start bit (1'b0)
- Data (LSB first or MSB, 8 bits)
- Even Parity bit
- One stop bit



2. Data Frame (in case of Parity is enabled & Parity Type is odd)

- One start bit (1'b0)
- Data (LSB first or MSB, 8 bits)
- Odd Parity bit
- One stop bit



3. Data Frame (in case of Parity is not Enabled)

- One start bit (1'b0)
- Data (LSB first or MSB, 8 bits)
- One stop bit

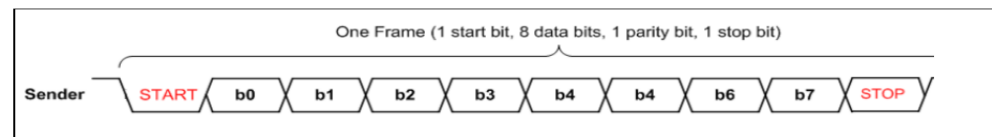


Figure 5: The three legal frame formats - even parity (EP), odd parity (OP), and no parity - all start with bit 0 and end with a stop bit of 1

03 DESIGN & ANALYSIS

3.1 Design Approach

UART_TX_TOP follows the specification's recommended structure exactly: a controlling FSM, a Serializer that registers and shifts out P_DATA one bit at a time, a Parity_Calc block that registers a single parity bit, and a 4-to-1 output Mux that picks which bit drives TX_OUT at each point in the frame. The FSM owns all timing decisions (when to register new data, when to enable the serializer, which Mux input to select); the other three blocks are simple, mostly combinational/registered helpers with no state-machine logic of their own. This separation means the frame format (start/data/parity/stop ordering) lives entirely in the FSM's state list and its muxSel case statement, while the data path blocks stay reusable and easy to verify in isolation.

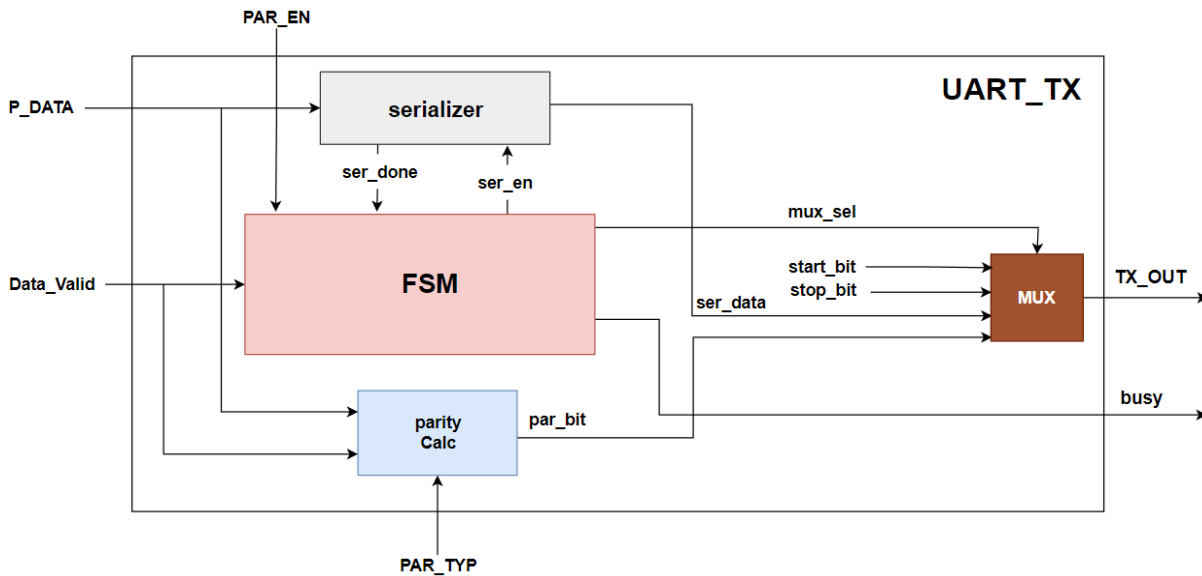


Figure 6: Recommended block diagram from the specification - matches the actual UART_TX_TOP structure: FSM, serializer, parity Calc and MUX

3.2 FSM States, Outputs and Transitions

UART_TX_FSM.sv defines five states. Because serEn, busy and regData are continuous functions of currentState (and dataValid for regData), and muxSel is set by a case on currentState, the design is effectively Moore-style: every output this cycle is fully determined by which state the FSM is already in.

State	Encoding	muxSel -> TX_OUT source	serEn	regData asserted when	Role
idle	3'b000	01 (line high)	0	0	Default/reset state; waits for dataValid
start	3'b001	00 (constant 0)	0	1 (only if entered from idle/stop with dataValid, sampled once)	Drives the start bit
data	3'b011	10 (serData)	1	0	Shifts out the 8 data bits, LSB first
parity	3'b010	11 (parBit)	0	0	Drives the parity bit (only reached if parEn was 1)
stop	3'b110	01 (line high)	0	1 (if dataValid already high)	Drives the stop bit; can chain directly into a new start

Table 2: FSM state outputs, derived directly from the combinational block in UART_TX_FSM.sv

Current State	Condition	Next State
idle	dataValid = 0	idle
idle	dataValid = 1	start
start	-	data (always, one cycle after start)
data	serDone = 0	data (hold, still shifting)
data	serDone = 1, parEn = 1	parity
data	serDone = 1, parEn = 0	stop
parity	-	stop (always, one cycle after parity)
stop	dataValid = 0	idle
stop	dataValid = 1	start (back-to-back frame, no idle gap)

Table 3: FSM next-state table, derived from the nextState case statement in UART_TX_FSM.sv

Two details are worth calling out because they are exactly what Scenarios 7 and 9 (Section 4) exist to test: (1) regData is only sampled while the FSM is in idle or stop, so a DATA_VALID pulse arriving during

start/data/parity is architecturally impossible to act on - it is simply not looked at; and (2) stop can go straight to start if DATA_VALID is already high, producing back-to-back frames with no idle gap and Busy never dropping.

3.3 Output Mux Selection

Mux_4X1 is instantiated as $Y = A[S]$ with $A = \{\text{parBitInternal}, \text{serDataInternal}, 1'b1, 1'b0\}$, i.e. $A[3]=\text{parBit}$, $A[2]=\text{serData}$, $A[1]=1$, $A[0]=0$. The FSM's muxSel therefore directly selects which signal reaches TX_OUT:

muxSel (S)	Selected input	Meaning on TX_OUT
00	$A[0] = 1'b0$	start bit
01	$A[1] = 1'b1$	idle / stop bit (line held high)
10	$A[2] = \text{serData}$	data bit currently on the serializer output
11	$A[3] = \text{parBit}$	registered parity bit

Table 4: Mux_4X1 selection table as wired in UART_TX_TOP.v

3.4 HDL Implementation

UART_TX_FSM.sv - state register, output logic and next-state logic:

```

module UART_TX_FSM (
    input parEn, serDone, dataValid, // inputs needed for FSM Transitions
    input CLK, RST,
    output reg serEn, busy, regData, // control signals of serializer and control unit
    output reg [1:0] muxSel // choose which bit for the TX_OUT
);
    // Define the states
    typedef enum logic [2:0] {
        idle = 'b000,
        start = 'b001,
        data = 'b011,
        parity = 'b010,
        stop = 'b110
    } state_t;
    state_t currentState, nextState;
    // Output Control Signals
    always @(*) begin
        serEn = (currentState == data); // to start from first bit and not to be shifted
        busy = (currentState != idle); // if it's not idle so it's working
        regData = dataValid && (currentState == idle || currentState == stop);
        muxSel = 'b0; // default mux selection
        // Set Mux Selection lines for TX_OUT
        case (currentState)
            start: muxSel = 'b00;
            data: muxSel = 'b10;
            parity: muxSel = 'b11;
            stop: muxSel = 'b01;
            default: muxSel = 'b01; // idle -> line idle high
        endcase
    end
    // Change the current state
    always @(posedge CLK or negedge RST) begin
        if(!RST) begin
            currentState <= idle;
        end
        else begin
            currentState <= nextState;
        end
    end
    // nextState Controlling
    always @(*) begin
        nextState = idle;
        case (currentState)
            idle: nextState = (dataValid) ? start : idle;
            start: nextState = data;
            data: nextState = (serDone) ? (parEn ? parity : stop) : data;
            parity: nextState = stop;
        endcase
    end
end

```

```

        stop: nextState = (dataValid) ? start : idle;
        default : nextState = idle;
    endcase
end
endmodule

```

The always @(*) block at the top computes every output straight from currentState (busy, serEn, muxSel) or from currentState plus dataValid (regData); the posedge CLK block advances currentState to nextState, with !RST forcing it back to idle asynchronously; the final always @(*) block is the next-state case table shown above.

Serializer.v - registers P_DATA once, then shifts one bit out per clock:

```

module Serializer (
    input [7:0] pData,
    input serEn, regData,
    input CLK, RST,
    output reg serData, serDone
);
    reg [7:0] pDataReg;
    reg [3:0] counter;
    wire counterMax = (counter == 7);
    always @(posedge CLK or negedge RST) begin
        if (!RST) pDataReg <= 0;
        else if (regData) pDataReg <= pData;
    end
    always @(*) begin
        serDone = counterMax;
        serData = pDataReg[counter];
    end
    always @(posedge CLK or negedge RST) begin
        if (!RST) begin
            counter <= 0;
        end
        else if (regData) begin
            counter <= 0;
        end
        else if (serEn) begin
            counter <= counter + 1;
        end
    end
end
endmodule //Serializer

```

pDataReg is only updated when regData is pulsed by the FSM (i.e. only when a new frame legitimately starts), which is exactly why Scenario 6 (P_DATA glitching on the bus with no DATA_VALID) and Scenario 7 (a spurious DATA_VALID while busy) cannot corrupt it. counter increments only while serEn is high (during the data state) and counterMax flags serDone once all 8 bits have been shifted, which the FSM uses to decide whether to move to parity or stop.

Parity_Calc.v - registers one parity bit per frame:

```

module Parity_Calc(
    input [7:0] pData,
    input parTyp, regData,
    input CLK, RST,
    output reg parBit
);
    always @(posedge CLK or negedge RST) begin
        if (!RST) parBit <= 0;
        else if (regData) begin
            parBit <= parTyp ? !(^pData) : (^pData);
        end
    end
end
endmodule

```

On the same regData pulse that loads pDataReg in the Serializer, Parity_Calc computes and registers parBit: a plain XOR-reduction of pData for even parity (parTyp=0), or its complement for odd parity (parTyp=1). Because this only happens on regData, parBit is stable for the rest of the frame regardless of what parTyp does afterwards - matching the testbench's discipline of holding parEn/parTyp constant for a whole frame.

Mux_4X1.v - 4-to-1 output multiplexer:

```
module Mux_4X1 (
    input [3:0] A,
    input [1:0] S,
    output reg Y
);
    always @(*) begin
        case (S)
            2'b00: Y = A[0];
            2'b01: Y = A[1];
            2'b10: Y = A[2];
            2'b11: Y = A[3];
        endcase
    end
endmodule //Mux_4X1
```

UART_TX_TOP.v - structural top level wiring the four blocks together:

```
module UART_TX_TOP (
    input [7:0] pData,
    input dataValid, parEn, parTyp,
    input CLK, RST,
    output txOut, busy
);
    wire serDoneInternal, serEnInternal, regDataInternal,
        serDataInternal,
        parBitInternal;
    wire [1:0] muxSelInternal;
    UART_TX_FSM FSM_Block (
        .parEn(parEn), .serDone(serDoneInternal), .dataValid(dataValid),
        .CLK(CLK), .RST(RST),
        .serEn(serEnInternal), .busy(busy), .regData(regDataInternal),
        .muxSel(muxSelInternal)
    );
    Serializer Serializer_Block (
        .pData(pData),
        .serEn(serEnInternal), .regData(regDataInternal),
        .CLK(CLK), .RST(RST),
        .serData(serDataInternal), .serDone(serDoneInternal)
    );
    Parity_Calc Parity_Calc_Block (
        .pData(pData),
        .parTyp(parTyp), .regData(regDataInternal),
        .CLK(CLK), .RST(RST),
        .parBit(parBitInternal)
    );
    Mux_4X1 Mux_4X1_Block (
        .A({parBitInternal, serDataInternal, 1'b1, 1'b0}),
        .S(muxSelInternal),
        .Y(txOut)
    );
endmodule //UART_TX_TOP
```

3.5 Static Lint Check

The design was additionally run through a SpyGlass lint/CDC/simulation rule deck (UART_Tx_SpyGlass.prj, moresimple.rpt). The report shows only 2 informational (not warning/error) messages: confirmation that UART_TX_TOP is correctly recognised as the top-level design unit, and a pointer to the elaboration summary

- no rule violations were reported across the erc, latch, lint, morelint, openmore, simulation, starc/starc2005 and timing policies.

04 SIMULATION & RESULTS

Verification approach: UART_TX_TOP_tb.sv is a self-checking testbench running at 200 MHz (5 ns period). A continuous monitor prints Busy/TX_OUT on every change. A reference calc_parity() function (independent of the RTL) and an expected_state_name() function build a per-bit expected sequence for each frame; send_and_check_frame() drives one byte and, every clock of the frame, cross-checks TX_OUT, Busy, the FSM's hierarchically-probed currentState, and the Serializer's serEn/serDone against that expected sequence, so a single test case is actually dozens of per-bit assertions rolled into one PASS/FAIL. Nine scenarios (test cases) are run in sequence, covering the normal frame formats plus five explicit corner cases the specification implies (reset behaviour, mid-frame reset, bus glitches, control-signal glitches, idle-line level, and back-to-back framing).

4.1 Test Case Summary

#	Scenario	Configuration	What it checks	Result
1	Scenario1_Reset	-	Asynchronous reset drives the FSM to idle, busy=0, txOut=1	PASSED
2	Scenario2_NoParity	pData=0xA5, parEn=0	One full frame with no parity bit (10-bit frame)	PASSED
3	Scenario3_EvenParity	pData=0xF3, parEn=1, parTyp=0	One full frame with even parity (11-bit frame)	PASSED
4	Scenario4_OddParity	pData=0x5A, parEn=1, parTyp=1	One full frame with odd parity (11-bit frame)	PASSED
5	Scenario5_ResetDuringFrame	pData=0x5A, mid-frame RST	Asynchronous reset asserted while transmitting must abort the frame back to idle	PASSED
6	Scenario6_PDataGlitch	pData=0x55, glitched to 0xAA mid-frame	P_DATA changing on the bus without a DATA_VALID pulse must not disturb the registered byte	PASSED
7	Scenario7_DataValidGlitchDuringFrame	pData=0x3C, parEn=1; spurious DATA_VALID pulse mid-frame	A stray DATA_VALID pulse while busy (not idle/stop) must be ignored	PASSED
8	Scenario8_IdleTxOutHigh	No data, dataValid=0	TX_OUT must stay high and Busy low while idle	PASSED
9	Scenario9_BackToBackNoIdle	pData=0x11 then 0x22, DATA_VALID pulsed during stop	Pulsing DATA_VALID while in stop must move straight to start with no idle gap (busy stays high)	PASSED

Table 5: Summary of all 9 self-checking test cases, from transcript.txt - 9/9 PASSED

4.2 Simulation Waveforms

Each test case was captured as two ModelSim screenshots that together show every probed signal (they are two halves of the same signal list and the same time window, split only because the full signal set does not fit one screen): the first ("-1") shows Top_Signals, FSM and Serializer; the second ("-2") shows the Parity group. Both are included for every test case below, with the specific behaviour each one demonstrates called out.

Test Case 1 - Scenario1_Reset (0 - 16 ns)

Purpose: confirm the asynchronous active-low reset forces the FSM to idle independent of the clock. In the capture, RST is driven low then released partway through the window; currentState/nextState read idle throughout, busy and txOut stay at their reset values, and testCount/passCount tick from 0 to 1 at the moment report_result() runs, confirming TEST 1 PASSED at 16000 ps as in the log.

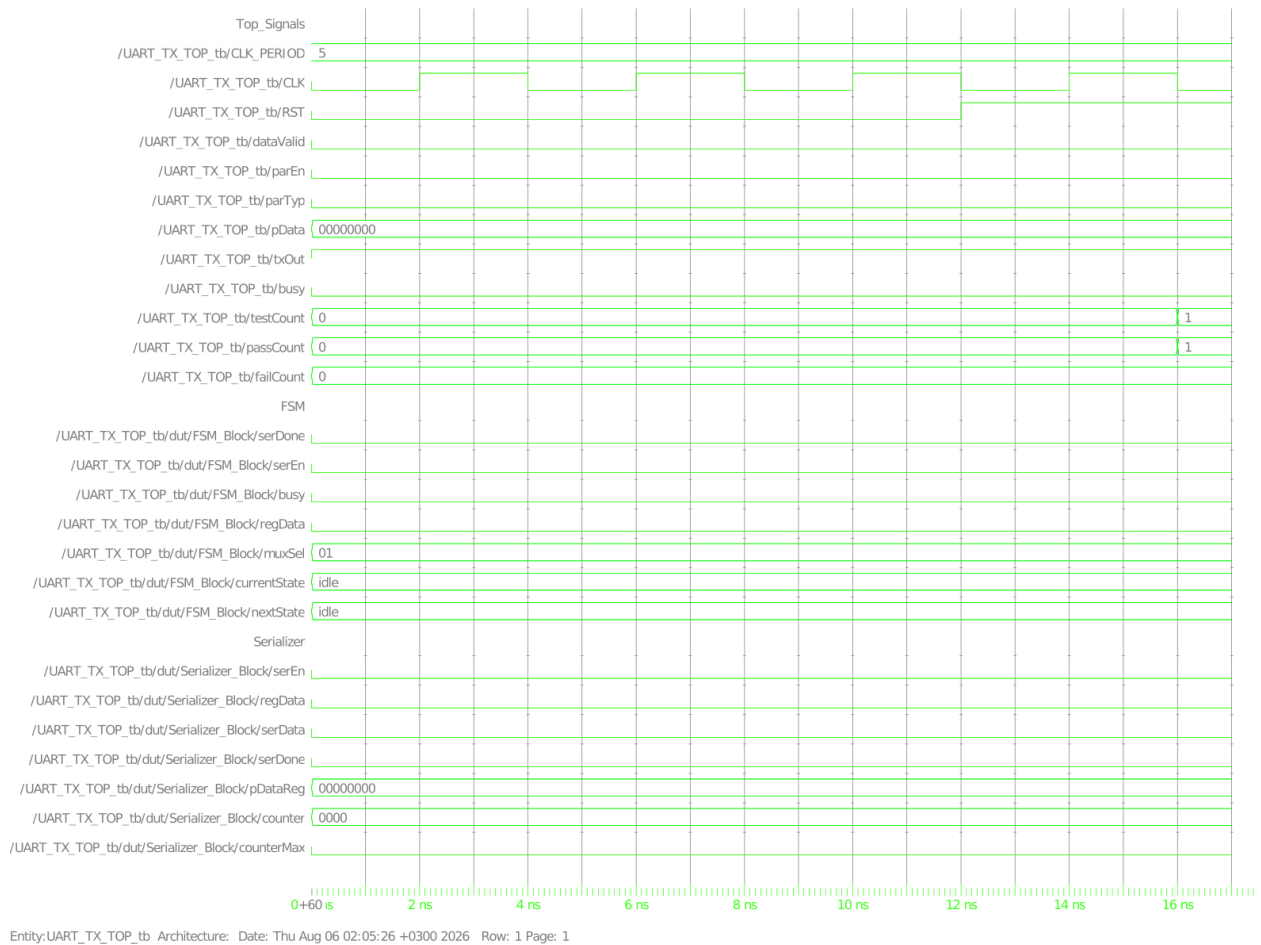


Figure 7: Test Case 1, view 1/2 - Top_Signals / FSM / Serializer showing RST assertion and currentState=idle

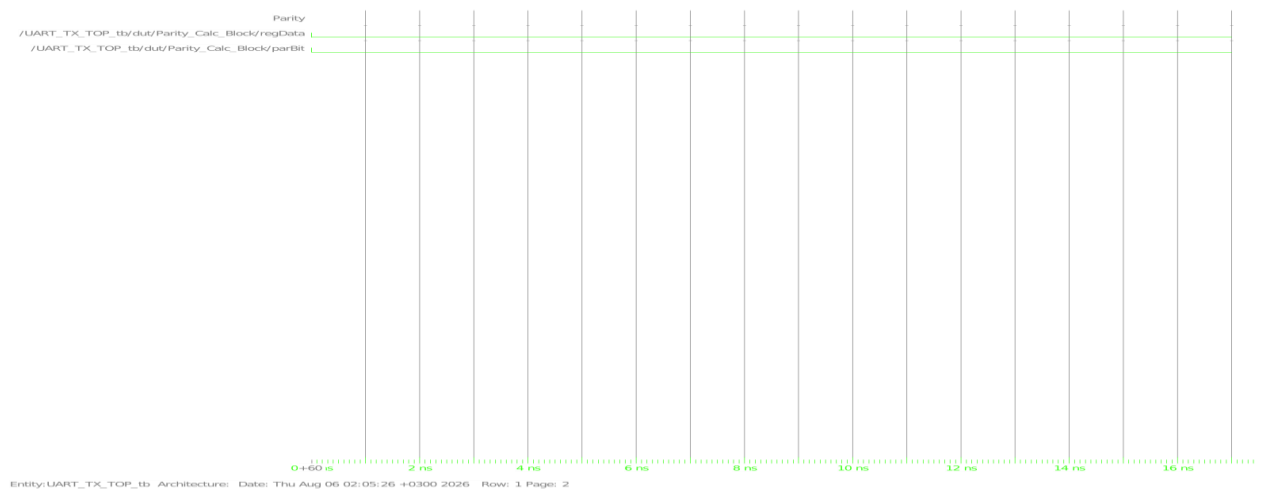


Figure 8: Test Case 1, view 2/2 - Parity group flat at 0, confirming no parity activity while reset/idle

Test Case 2 - Scenario2_NoParity (16 - 64 ns), pData=8'hA5, parEn=0

Purpose: verify one complete 10-bit frame (start + 8 data + stop, no parity) with parEn=0. The capture shows pData loaded as 10100101 (0xA5), currentState stepping idle -> start -> data -> stop -> idle, muxSel following 01 -> 00 -> 10 -> 01, and Serializer counter counting 0000 through 1000 while serEn is high, exactly matching the 10-bit, no-parity frame length required by the specification.

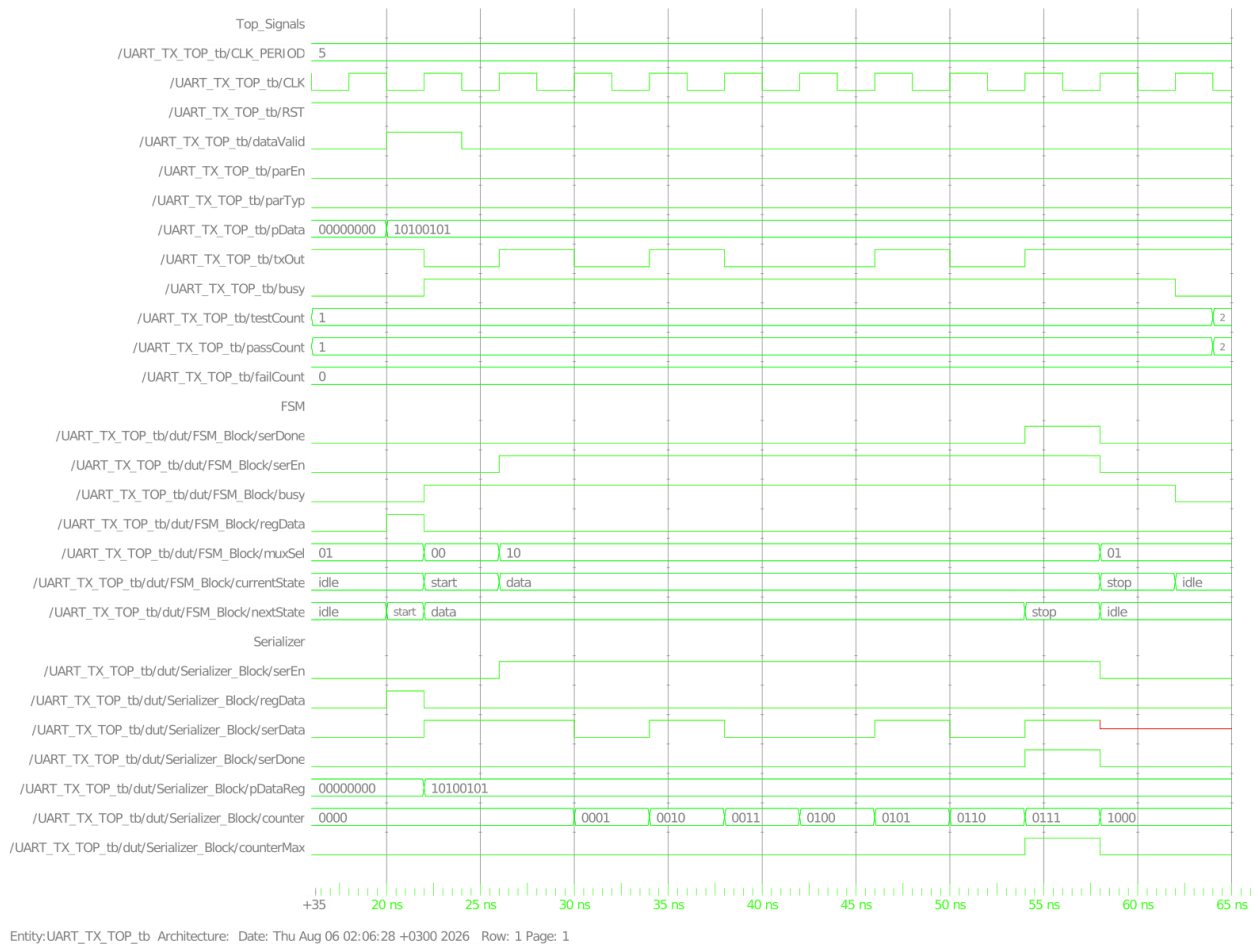


Figure 9: Test Case 2, view 1/2 - full no-parity frame, FSM state sequence idle-start-data-stop-idle

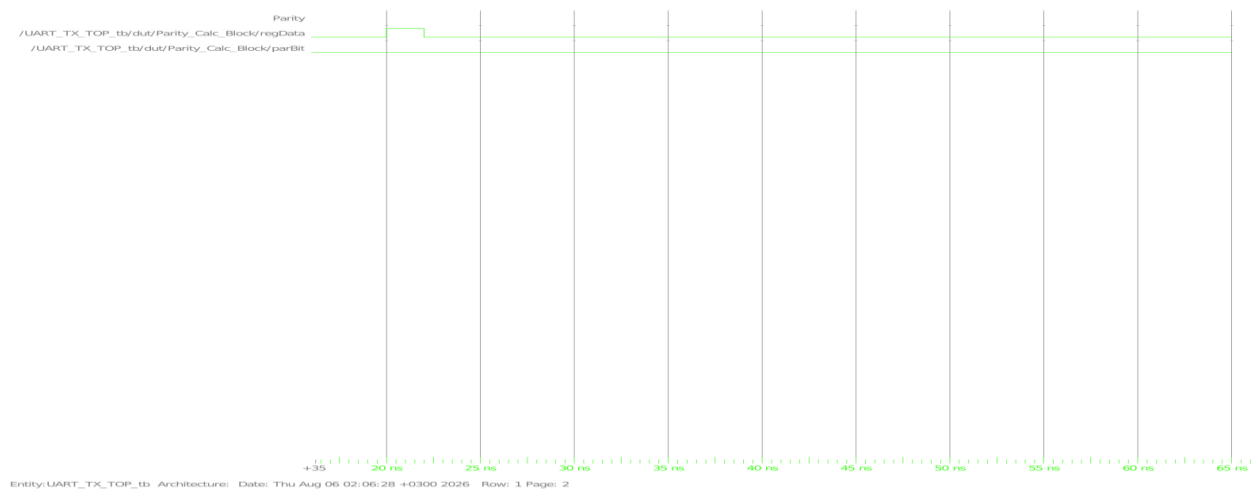


Figure 10: Test Case 2, view 2/2 - Parity group stays low throughout, confirming Parity_Calc is not exercised when parEn=0

Test Case 3 - Scenario3_EvenParity (64 - 116 ns), pData=8'hF3, parEn=1, parTyp=0

Purpose: verify an 11-bit frame (start + 8 data + even parity + stop) with the FSM's parity state actually reached. The Parity view shows Parity_Calc_Block/regData pulsing once as the frame is loaded and parBit settling to 0 (0xF3 = 11110011 has six set bits, an even count, so XOR = 0 as expected for even parity) and holding that value for the rest of the frame, which is what the Mux serialises out during muxSel=11.

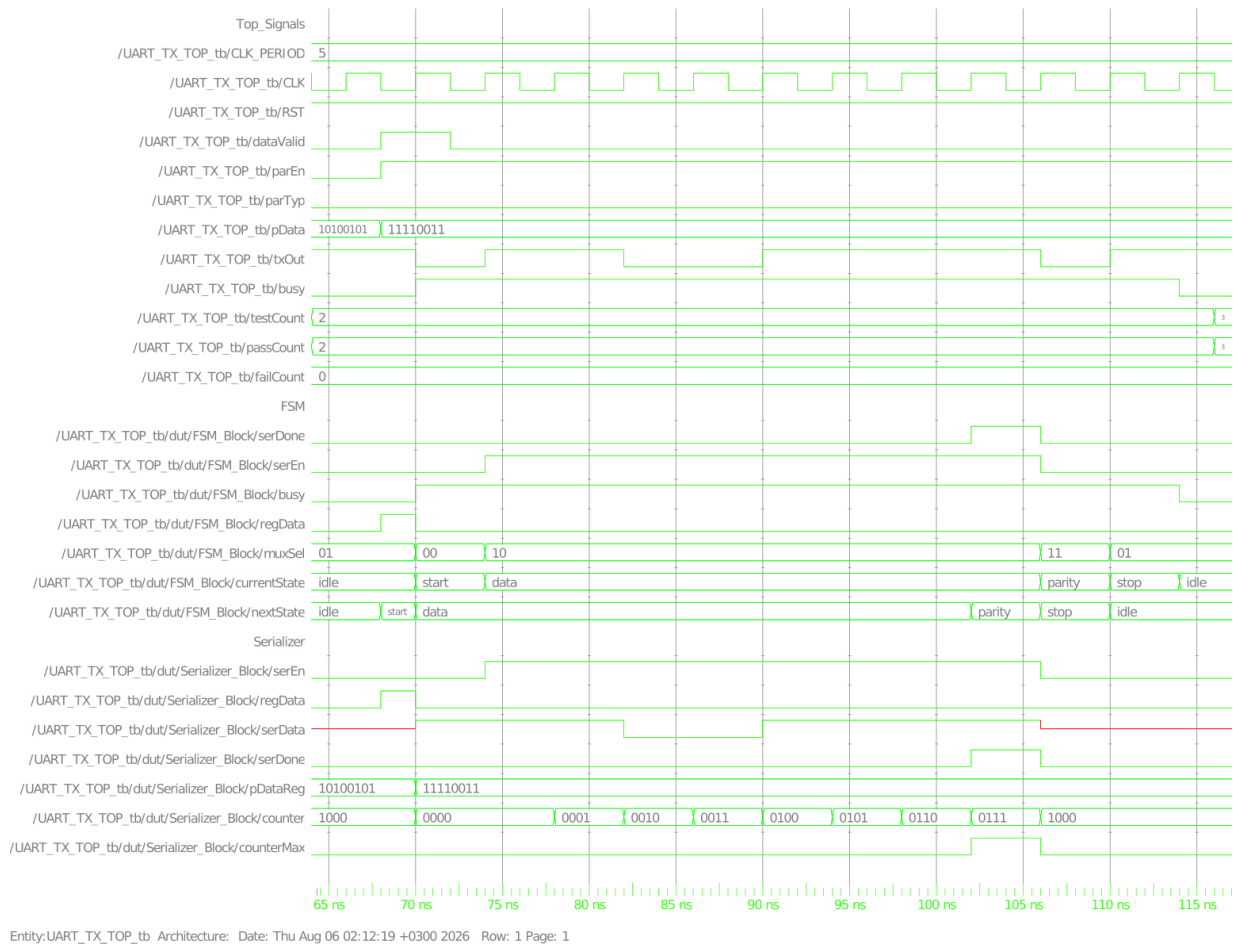


Figure 11: Test Case 3, view 1/2 - state sequence idle-start-data-parity-stop-idle (parity state now reached)

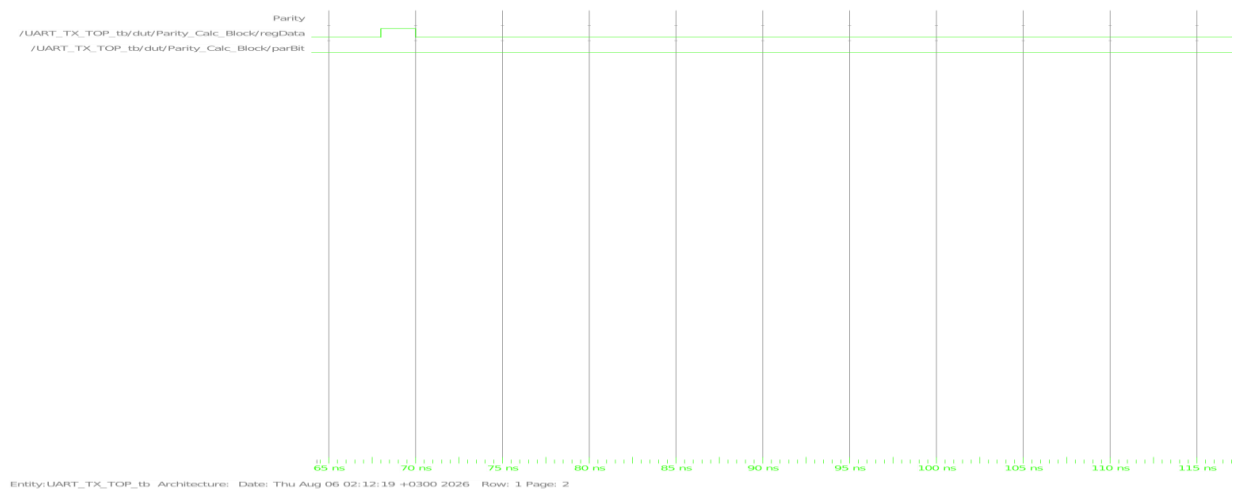


Figure 12: Test Case 3, view 2/2 - Parity_Calc regData pulses once, parBit registers and holds at 0 (even parity of 0xF3)

Test Case 4 - Scenario4_OddParity (116 - 168 ns), pData=8'h5A, parEn=1, parTyp=1

Purpose: verify the same 11-bit framing with the opposite parity polarity. The capture again shows the FSM passing through the parity state (muxSel=11) between data and stop, this time with parTyp=1 selecting the odd-parity branch of Parity_Calc's ternary; the Parity view shows parBit registering once on the regData pulse and holding for the remainder of the frame.



Figure 13: Test Case 4, view 1/2 - identical framing shape to Test Case 3, confirming parTyp only changes the parity value, not the frame structure

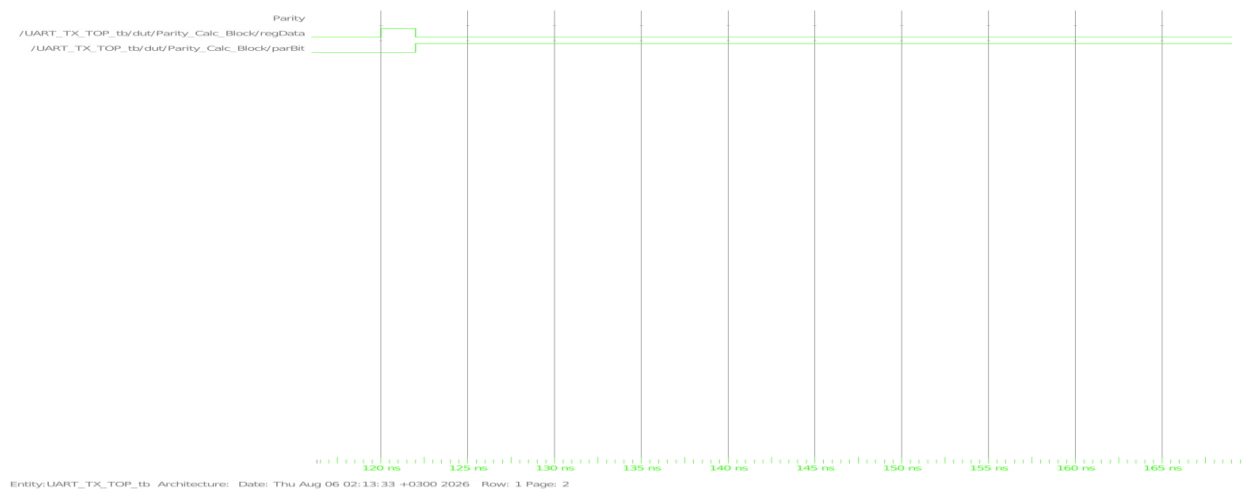


Figure 14: Test Case 4, view 2/2 - Parity_Calc registers the odd-parity bit for 0x5A

Test Case 5 - Scenario5_ResetDuringFrame (168 - 196 ns)

Purpose: verify RST is honoured even mid-transmission, not just before a frame starts. The waveform shows a frame beginning normally (pDataReg loads 01011010 / 0x5A), then RST is asserted while the FSM is in the data state; currentState, busy and pDataReg all snap back to idle/0 within the same delta (the design's reset is asynchronous, so this does not wait for a clock edge), and Serializer counter resets to 0000. This directly

exercises the specification's asynchronous active-low reset requirement under the harder condition of an in-flight frame.

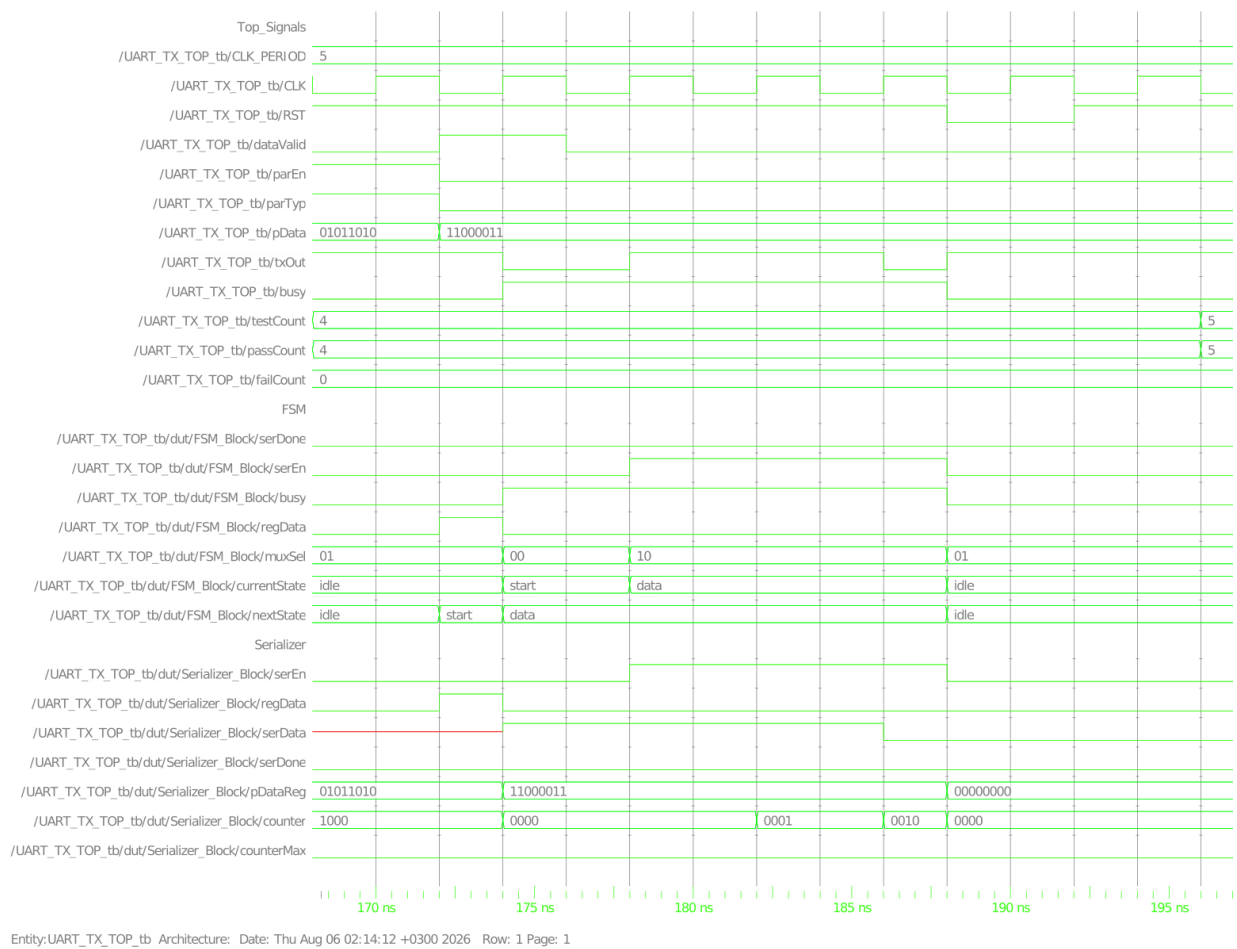


Figure 15: Test Case 5, view 1/2 - RST asserted mid-frame; currentState, busy and pDataReg all clear immediately

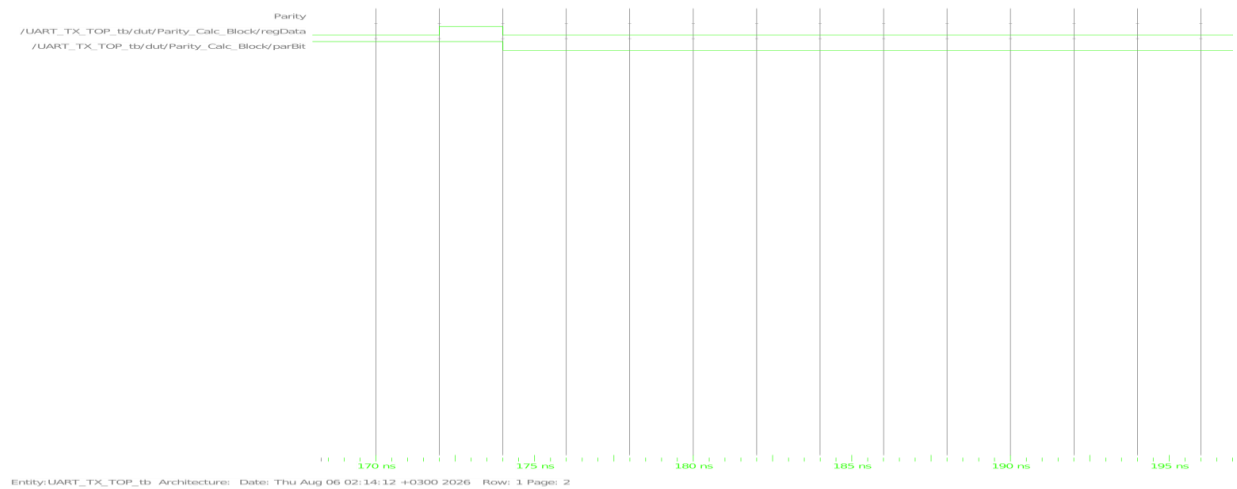


Figure 16: Test Case 5, view 2/2 - Parity group unaffected/inactive (parEn=0 for this scenario)

Test Case 6 - Scenario6_PDataGlitch (196 - 244 ns)

Purpose: verify that P_DATA changing on the bus without a fresh DATA_VALID pulse cannot corrupt an in-flight frame. pData is loaded as 01010101 (0x55); partway through the frame the testbench drives pData to

10101010 (0xAA) directly on the bus (no dataValid). The capture shows Serializer_Block/pDataReg staying at 01010101 for the entire frame - the glitch on the top-level pData wire never reaches the internal register because regData (and therefore the load enable) never pulses again until the next legitimate frame.

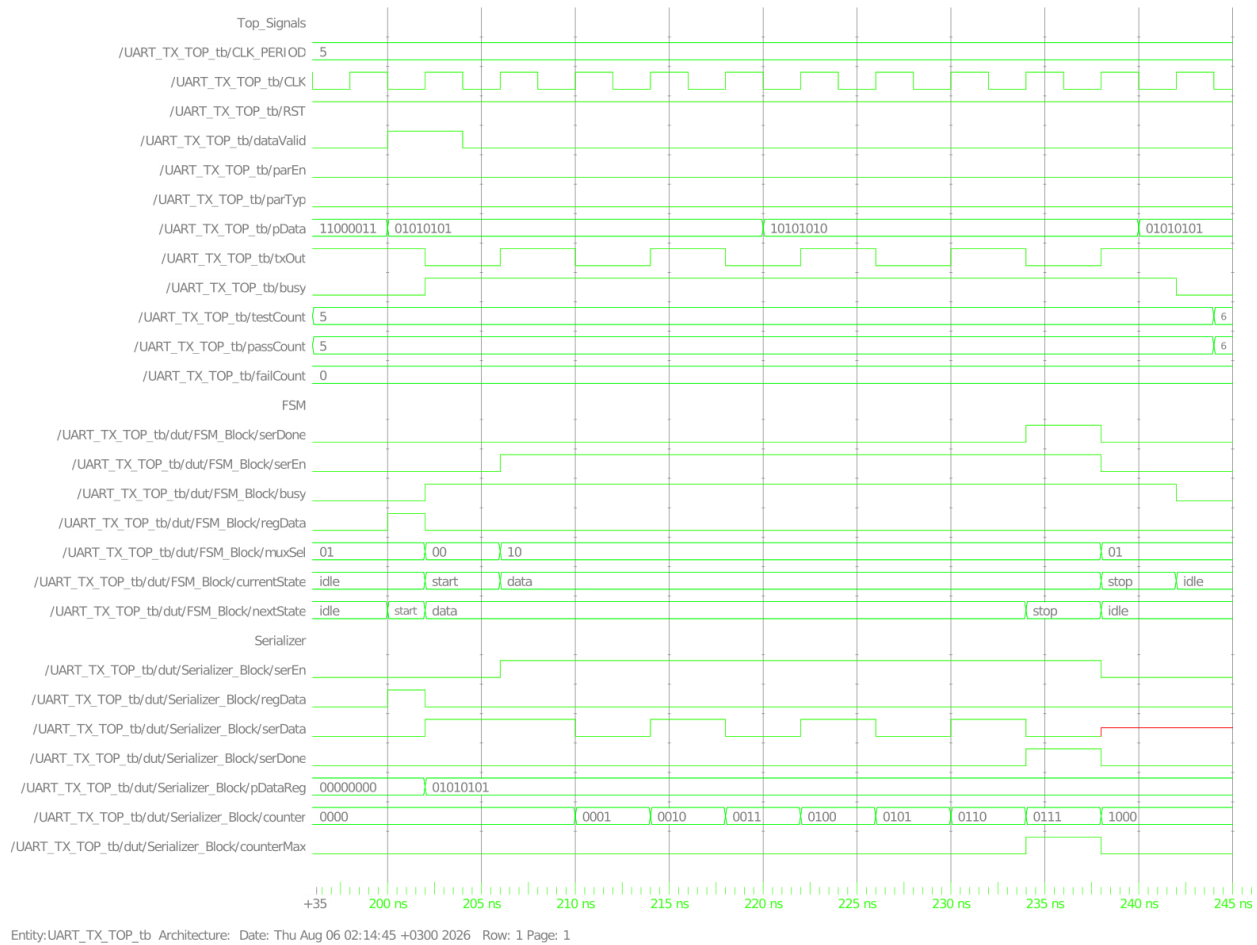


Figure 17: Test Case 6, view 1/2 - pData bus glitches to 0xAA mid-frame but pDataReg (and therefore TX_OUT) is unaffected

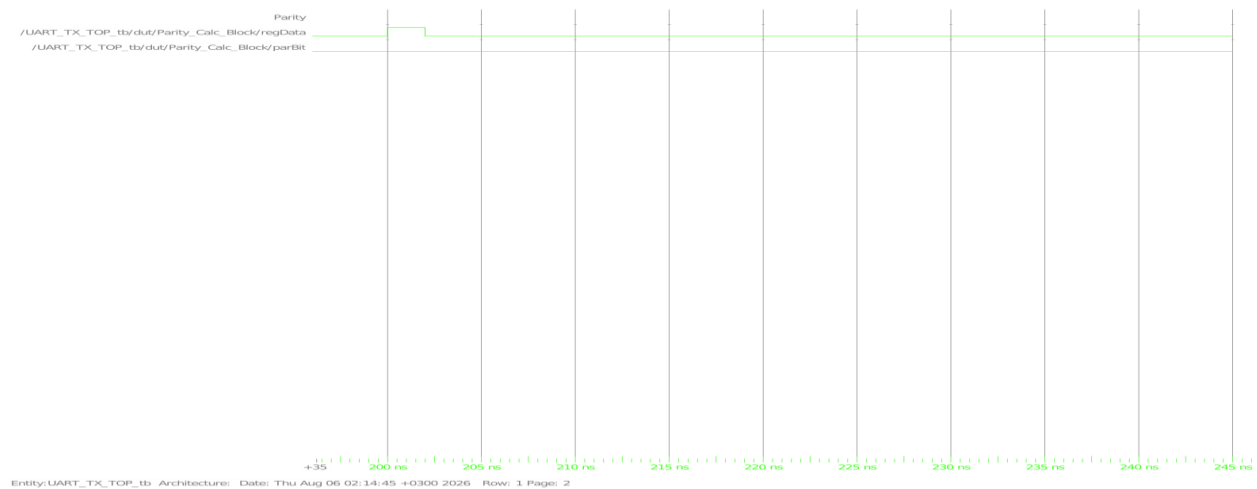


Figure 18: Test Case 6, view 2/2 - Parity group inactive (parEn=0 for this scenario)

Test Case 7 - Scenario7_DataValidGlitchDuringFrame (244 - 296 ns)

Purpose: verify a spurious DATA_VALID pulse while the FSM is busy (not idle/stop) is ignored, per the specification's "UART_TX couldn't accept any data on P_DATA during UART_TX processing, however Data_Valid get high" requirement. pData=0x3C is loaded with parEn=1 (even parity); partway through the data state the testbench pulses dataValid again with a different byte (0x99) staged on pData. The capture shows a second, unmistakable dataValid pulse appearing mid-frame, yet Serializer_Block/pDataReg never changes from 00111100 and the FSM proceeds start-data-parity-stop-idle exactly as if the glitch had not happened - because regData is only ever asserted while currentState is idle or stop (Table 2), so the FSM structurally cannot act on dataValid during data.

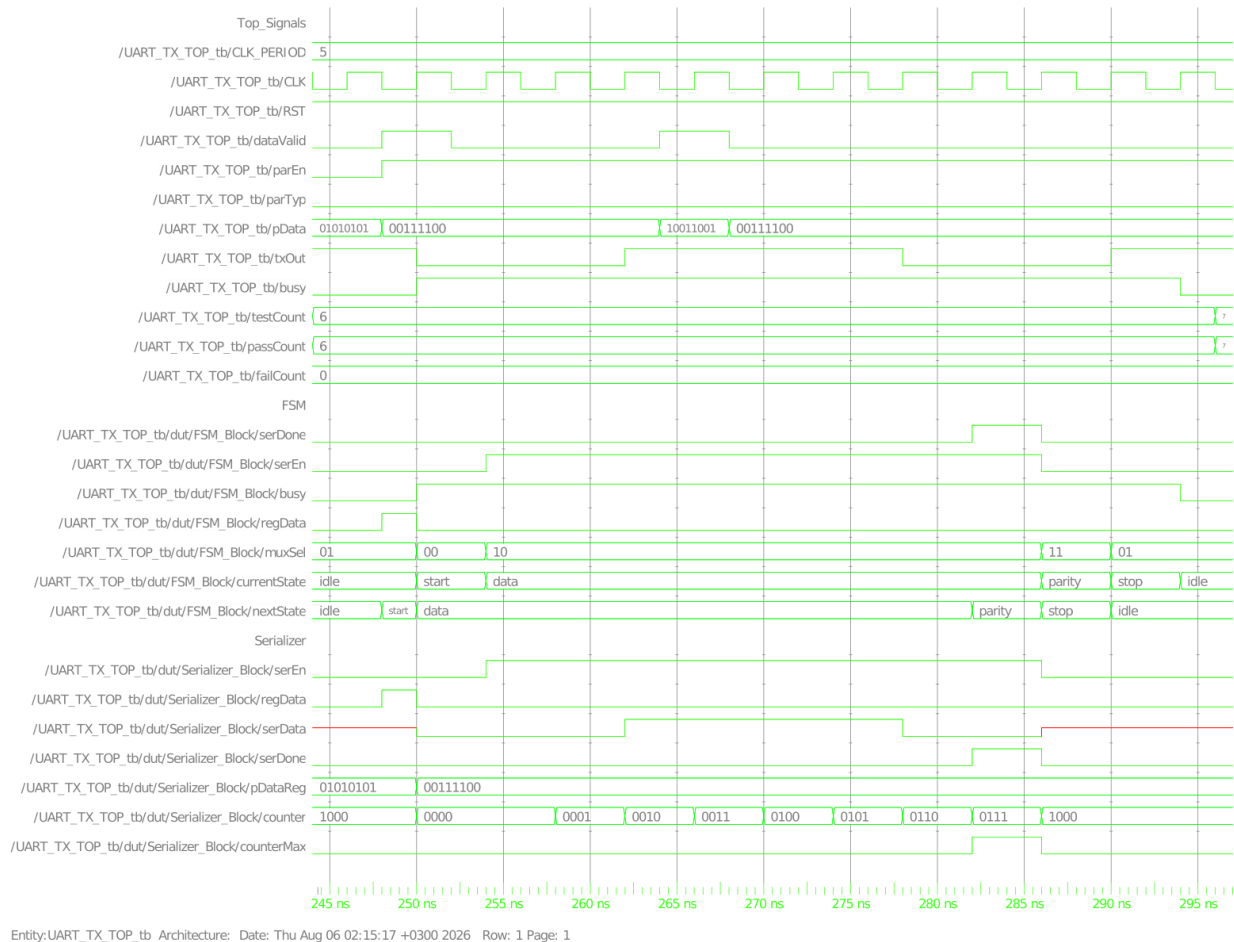


Figure 19: Test Case 7, view 1/2 - a second dataValid pulse fires mid-frame (visible on the dataValid trace) but pDataReg and the FSM state sequence are unaffected

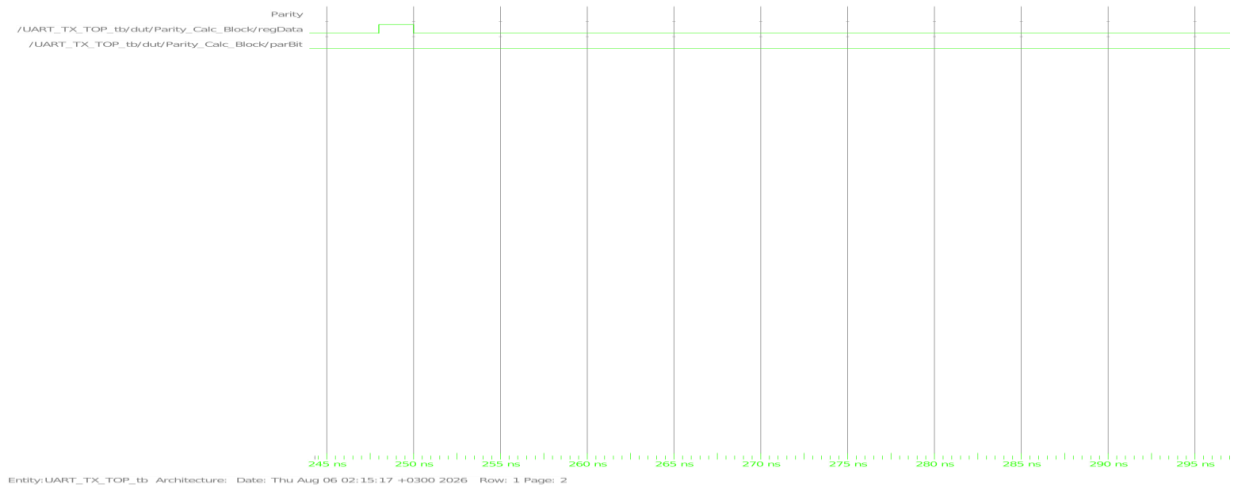


Figure 20: Test Case 7, view 2/2 - Parity_Calc still only latches once, on the legitimate load, not on the glitch

Test Case 8 - Scenario8_IdleTxOutHigh (296 - 316 ns)

Purpose: verify the specification's "TX_OUT is high in the IDLE case" requirement in isolation, with no frame in flight. The capture shows dataValid held low for the whole window, currentState/nextState remaining idle, busy remaining low, and muxSel fixed at 01 (which Table 4 shows selects the constant-1 Mux input) - confirming TX_OUT is driven high purely by the idle-state Mux selection, not by some default/undriven value.

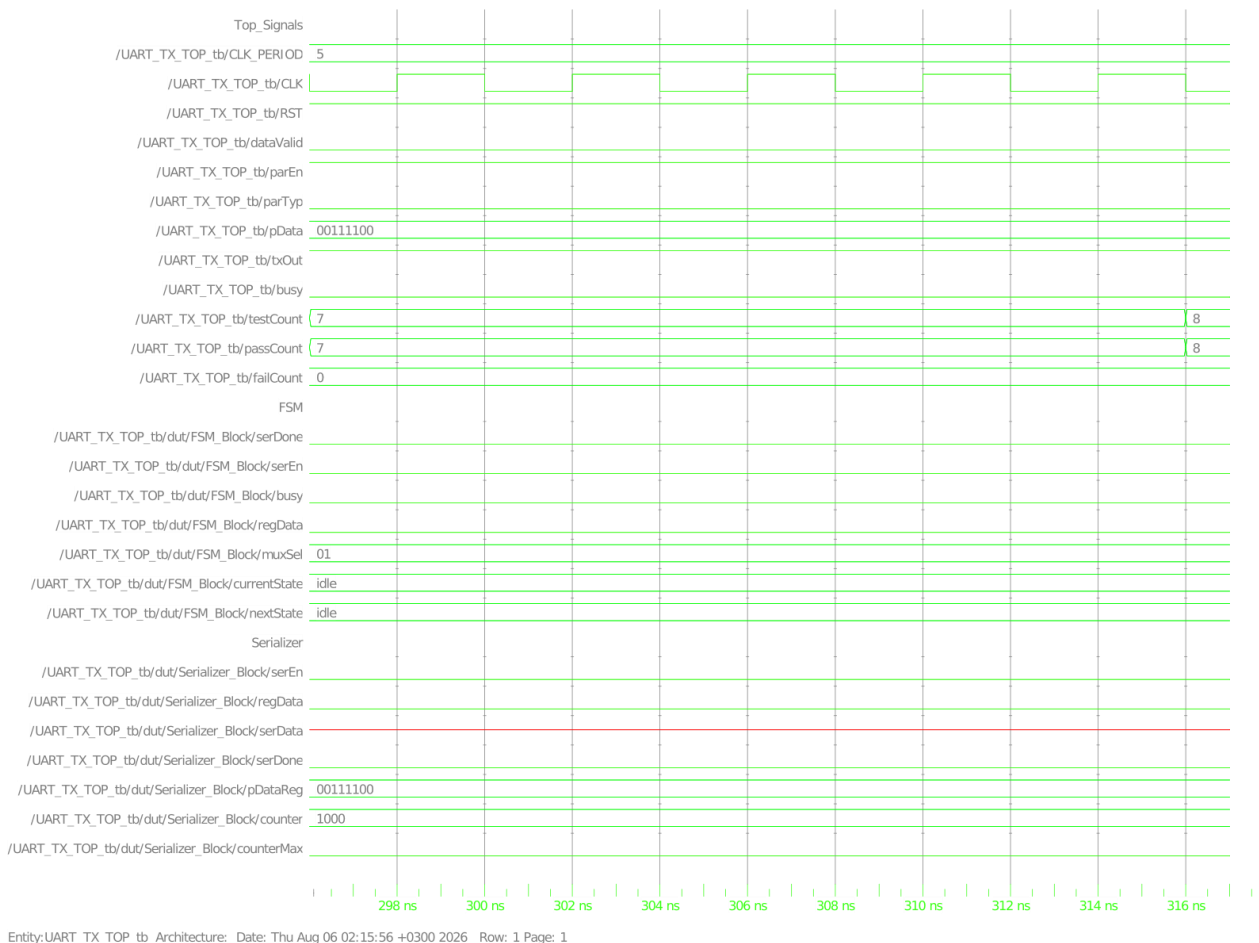


Figure 21: Test Case 8, view 1/2 - FSM parked in idle, muxSel=01, busy=0 for the full 20 ns window

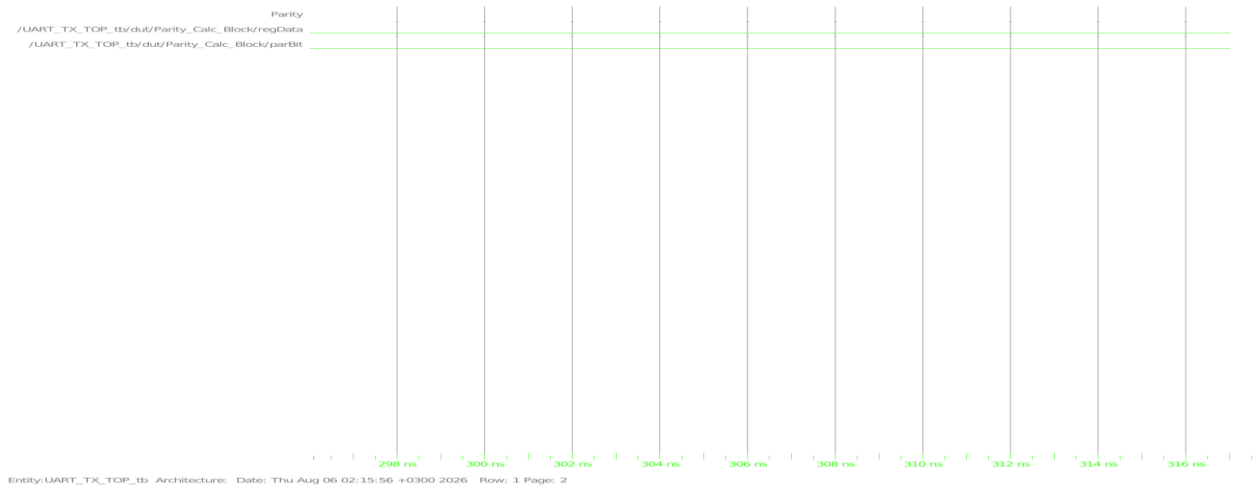


Figure 22: Test Case 8, view 2/2 - Parity group inactive, as expected with no frame in progress

Test Case 9 - Scenario9_BackToBackNoIdle (316 - 404 ns)

Purpose: verify that pulsing DATA_VALID while the FSM is in stop starts the next frame immediately, with no idle gap and Busy never dropping. Frame 1 (pData=0x11) transmits normally; while currentState reads stop for frame 1's stop bit, the testbench pulses dataValid again with pData=0x22. The capture shows currentState going directly from stop to start (idle is skipped entirely) and busy staying asserted (high) across the boundary between the two frames - exactly the back-to-back behaviour the next-state table (Table 3, stop with dataValid=1 -> start) predicts.

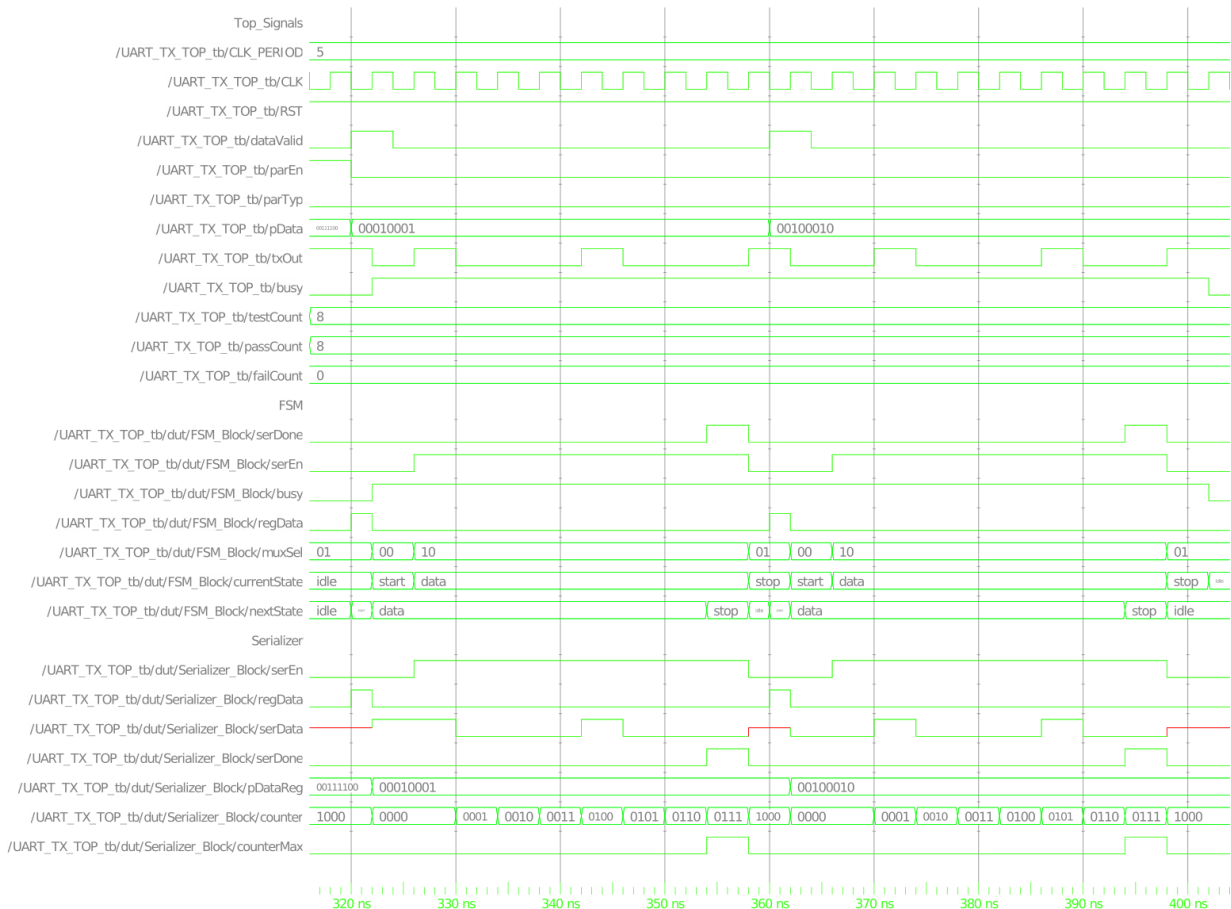


Figure 23: Test Case 9, view 1/2 - two consecutive frames (0x11 then 0x22) with currentState transitioning stop -> start directly and busy never returning to 0 between them

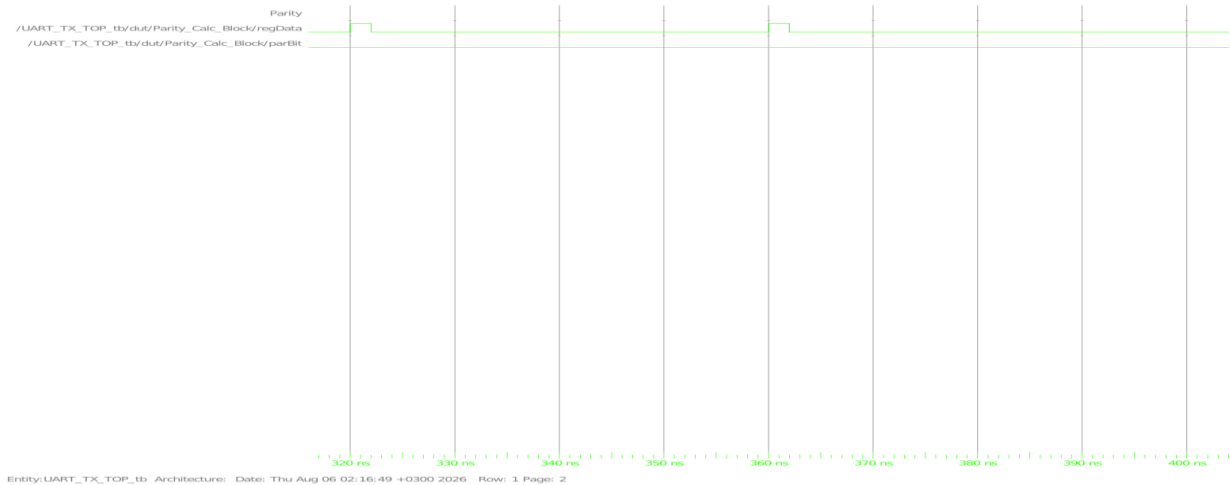


Figure 24: Test Case 9, view 2/2 - Parity group inactive (parEn=0 for both frames in this scenario)

4.3 Full Simulation Log

Console transcript produced by UART_TX_TOP_tb.sv - every MONITOR line, every scenario START/END marker, every per-test PASSED result, and the final tally (9 run, 9 PASSED, 0 FAILED):

```
=====
UART_TX_TOP Self-Checking Testbench
Clock: 200MHz (Period = 5 ns)
=====
[0] --- Scenario1_Reset START ---
[0] MONITOR: busy=x txOut=1
[0] MONITOR: busy=0 txOut=1
[16000] --- Scenario1_Reset END ---
[16000] TEST 1 - Scenario1_Reset : PASSED
[16000] --- Scenario2_NoParity START (pData=8'ha5 parEn=0 parTyp=0) ---
[22000] MONITOR: busy=1 txOut=1
[22000] MONITOR: busy=1 txOut=0
[26000] MONITOR: busy=1 txOut=1
[30000] MONITOR: busy=1 txOut=0
[34000] MONITOR: busy=1 txOut=1
[38000] MONITOR: busy=1 txOut=0
[46000] MONITOR: busy=1 txOut=1
[50000] MONITOR: busy=1 txOut=0
[54000] MONITOR: busy=1 txOut=1
[62000] MONITOR: busy=0 txOut=1
[64000] --- Scenario2_NoParity END ---
[64000] TEST 2 - Scenario2_NoParity : PASSED
[64000] --- Scenario3_EvenParity START (pData=8'hf3 parEn=1 parTyp=0) ---
[70000] MONITOR: busy=1 txOut=1
[70000] MONITOR: busy=1 txOut=0
[74000] MONITOR: busy=1 txOut=1
[82000] MONITOR: busy=1 txOut=0
[90000] MONITOR: busy=1 txOut=1
[106000] MONITOR: busy=1 txOut=0
[110000] MONITOR: busy=1 txOut=1
[114000] MONITOR: busy=0 txOut=1
[116000] --- Scenario3_EvenParity END ---
[116000] TEST 3 - Scenario3_EvenParity : PASSED
[116000] --- Scenario4_OddParity START (pData=8'h5a parEn=1 parTyp=1) ---
[122000] MONITOR: busy=1 txOut=1
[122000] MONITOR: busy=1 txOut=0
[130000] MONITOR: busy=1 txOut=1
[134000] MONITOR: busy=1 txOut=0
[138000] MONITOR: busy=1 txOut=1
[146000] MONITOR: busy=1 txOut=0
[150000] MONITOR: busy=1 txOut=1
[154000] MONITOR: busy=1 txOut=0
[158000] MONITOR: busy=1 txOut=1
[166000] MONITOR: busy=0 txOut=1
[168000] --- Scenario4_OddParity END ---
[168000] TEST 4 - Scenario4_OddParity : PASSED
```

```

[168000] --- Scenario5_ResetDuringFrame START ---
[174000] MONITOR: busy=1 txOut=1
[174000] MONITOR: busy=1 txOut=0
[178000] MONITOR: busy=1 txOut=1
[186000] MONITOR: busy=1 txOut=0
[188000] MONITOR: busy=0 txOut=0
[188000] MONITOR: busy=0 txOut=1
[196000] --- Scenario5_ResetDuringFrame END ---
[196000] TEST 5 - Scenario5_ResetDuringFrame : PASSED
[196000] --- Scenario6_PDataGlitch START ---
[202000] MONITOR: busy=1 txOut=1
[202000] MONITOR: busy=1 txOut=0
[206000] MONITOR: busy=1 txOut=1
[210000] MONITOR: busy=1 txOut=0
[214000] MONITOR: busy=1 txOut=1
[218000] MONITOR: busy=1 txOut=0
[222000] MONITOR: busy=1 txOut=1
[226000] MONITOR: busy=1 txOut=0
[230000] MONITOR: busy=1 txOut=1
[234000] MONITOR: busy=1 txOut=0
[238000] MONITOR: busy=1 txOut=1
[242000] MONITOR: busy=0 txOut=1
[244000] --- Scenario6_PDataGlitch END ---
[244000] TEST 6 - Scenario6_PDataGlitch : PASSED
[244000] --- Scenario7_DataValidGlitchDuringFrame START ---
[250000] MONITOR: busy=1 txOut=1
[250000] MONITOR: busy=1 txOut=0
[262000] MONITOR: busy=1 txOut=1
[278000] MONITOR: busy=1 txOut=0
[290000] MONITOR: busy=1 txOut=1
[294000] MONITOR: busy=0 txOut=1
[296000] --- Scenario7_DataValidGlitchDuringFrame END ---
[296000] TEST 7 - Scenario7_DataValidGlitchDuringFrame : PASSED
[296000] --- Scenario8_IdleTxOutHigh START ---
[316000] --- Scenario8_IdleTxOutHigh END ---
[316000] TEST 8 - Scenario8_IdleTxOutHigh : PASSED
[316000] --- Scenario9_BackToBackNoIdle START ---
[322000] MONITOR: busy=1 txOut=1
[322000] MONITOR: busy=1 txOut=0
[326000] MONITOR: busy=1 txOut=1
[330000] MONITOR: busy=1 txOut=0
[342000] MONITOR: busy=1 txOut=1
[346000] MONITOR: busy=1 txOut=0
[358000] MONITOR: busy=1 txOut=1
[362000] MONITOR: busy=1 txOut=0
[370000] MONITOR: busy=1 txOut=1
[374000] MONITOR: busy=1 txOut=0
[386000] MONITOR: busy=1 txOut=1
[390000] MONITOR: busy=1 txOut=0
[398000] MONITOR: busy=1 txOut=1
[402000] MONITOR: busy=0 txOut=1
[404000] --- Scenario9_BackToBackNoIdle END ---
[404000] TEST 9 - Scenario9_BackToBackNoIdle : PASSED
=====
TEST SUMMARY: 9 run, 9 PASSED, 0 FAILED
=====
** Note: $finish : UART_TX_TOP_tb.sv(510)
Time: 404 ns Iteration: 1 Instance: /UART_TX_TOP_tb

```

Log 1: Full ModelSim transcript for UART_TX_TOP_tb

05 DISCUSSION

All 9 self-checking scenarios passed, and because `send_and_check_frame()` re-validates TX_OUT, Busy, FSM state and Serializer `serEn/serDone` on every single clock of every frame (not just at the end), each PASSED result in the log represents dozens of underlying bit-level and state-level assertions, not one coarse check. The four framing scenarios (2-4, plus the no-parity path exercised throughout 5/6/8/9) confirm both frame lengths (10-bit without parity, 11-bit with parity) and both parity polarities compute the correct bit and place it in the correct slot via `muxSel=11`. The five robustness scenarios (1, 5, 6, 7, 9) map directly onto specification requirements that are easy to satisfy on the happy path but easy to get wrong at the edges: asynchronous reset (1 and 5, including reset asserted mid-frame - a case many naive designs only handle before a frame starts), refusing new P_DATA while busy despite bus glitches (6) or a spurious DATA_VALID (7), and correctly

chaining frames back-to-back without an idle gap when the next DATA_VALID arrives during stop (9). The waveforms corroborate the transcript rather than merely restating it - in each case the specific internal signal (pDataReg, parBit, currentState, or the dataValid pulse itself) that the scenario is designed to stress is visibly doing the right thing. The one design property worth flagging for future work is that DATA_VALID pulses that arrive while idle or stop are, by construction, always accepted - a true back-pressure/handshake (e.g. a ready output) is not part of this specification and was not tested.

06 CONCLUSION

A UART transmitter was implemented as four cooperating Verilog/SystemVerilog blocks (UART_TX_FSM, Serializer, Parity_Calc, Mux_4X1) wired together in UART_TX_TOP, matching the specification's recommended architecture. A 9-scenario, per-bit self-checking testbench running at 200 MHz confirmed correct framing for both parity-enabled (even and odd) and parity-disabled frames, correct asynchronous reset behaviour including mid-frame reset, correct rejection of P_DATA bus glitches and spurious DATA_VALID pulses while busy, correct idle-line behaviour, and correct back-to-back frame chaining with no idle gap. A SpyGlass lint pass additionally reported zero rule violations across the erc/latch/lint/morelint/openmore/simulation/starc/timing policies.